

```
"""
```

LLM-Assisted Content Analysis — AI Leadership Role Job Ads

```
=====
```

Reads all job ads from a multi-sheet Excel file, sends each full_text to the Anthropic API with the coding prompt, and writes the structured coding results to an output Excel file.

Usage:

```
python ai_leadership_coding.py
```

Requirements:

```
pip install anthropic openpyxl pandas
```

Configuration:

Set your API key in the ANTHROPIC_API_KEY variable below,
or export it as an environment variable before running.

```
"""
```

```
import os
```

```
import json
```

```
import time
```

```
import traceback
```

```
import anthropic
```

```
import openpyxl
```

```
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
```

```
from openpyxl.utils import get_column_letter
```

```
from datetime import datetime
```

```
# =====
```

CONFIGURATION — edit these paths and settings

=====

Path to your combined Excel dataset (all role sheets in one file)

INPUT_FILE = "Dataset.xlsx"

Which sheets to process — list all role sheets, skip overview sheets

SHEETS_TO_PROCESS = [

"AI Manager",

"Head of AI",

"Chief AI Officer",

"AI Lead",

"CTO",

"CIO",

"CDO",

]

Output file path

OUTPUT_FILE = f"coding_output_{datetime.now().strftime('%Y%m%d_%H%M')}.xlsx"

Your Anthropic API key — set here or as environment variable

ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY", "sk-ant-api03-CYNkTphVkDMRFEzhpQfyYfBUsgYhJL4jtITwwAA8y_QOrNOVMV6VCA3RrVJg_wGbJpYHOeZl5K7ypoRUfXxtSw-C7UebgAA")

Model to use

MODEL = "claude-sonnet-4-6"

Max tokens for response — increase if ads are very long

MAX_TOKENS = 4000

Delay between API calls in seconds — prevents rate limiting

API_DELAY = 1.5

If True, skip ads where full_text is empty or very short (<100 chars)

SKIP_EMPTY = True

Resume from a specific ad index if a previous run was interrupted

Set to 0 to start from the beginning

RESUME_FROM_INDEX = 0

=====

PROMPT TEMPLATE

=====

SYSTEM_PROMPT = """You are a research assistant supporting a scientific content analysis of job advertisements for AI leadership roles and technology leadership roles. Your task is to apply a theory-derived codebook to job ad texts and return structured coding decisions.

You operate with precision and consistency. You follow the coding rules exactly as specified. You do not interpret beyond what the text says. You do not add codes that are not supported by explicit textual evidence. You always return valid JSON."""

USER_PROMPT_TEMPLATE = """## RESEARCH CONTEXT

You are assisting with an academic content analysis study examining how organizations configure dedicated AI leadership roles (AI Manager, Head of AI, Chief AI Officer, AI Lead) compared to traditional technology leadership roles (CTO, CIO, CDO).

The analysis uses a two-layer codebook:

- Layer 1: Governance tension codes (GT) — derived from IS/IT governance theory
- Layer 2: Skill/competence codes (SK) — derived from Todd et al. (1995)

Your task is to read a complete job advertisement, identify all passages that are relevant for coding, and return a structured coding output for each identified passage.

YOUR TASK

Read the full job advertisement provided below. Identify every passage — a sentence, a group of sentences, or a bullet point cluster — that meets the AI specificity requirement and maps to at least one code in the codebook. Extract each such passage as a text segment and assign the appropriate codes.

A passage is worth extracting when it contains a meaningful reference to AI, ML, GenAI, or direct equivalents AND maps to at least one GT or SK code. Do not extract passages that are purely administrative (salary, benefits, equal opportunity statements, company boilerplate) or that contain no AI-relevant governance or competence content.

If the entire job advertisement contains no AI-specific content, return a single entry with GT_Code: NONE, SK_Code: NONE, and explain in the Rationale why no codes apply.

CRITICAL RULE: AI SPECIFICITY REQUIREMENT

Before coding any passage, check whether it meets the AI specificity requirement. A passage is eligible for coding ONLY if it contains a meaningful reference to AI, ML, GenAI, large language models, or direct equivalents (e.g., machine learning, artificial intelligence, agentic AI).

Two types of AI specificity are recognized:

1. LEXICAL SPECIFICITY: AI, ML, GenAI, or a direct equivalent is explicitly named within the passage itself. This is the default and preferred basis for coding.

2. CONTEXTUAL SPECIFICITY: AI is not named in the passage, but the passage is embedded within a clearly AI-dedicated section of the job ad (e.g., a section titled "AI Governance" or "AI Strategy") and the passage cannot be meaningfully read as referring to anything other than AI. Contextual specificity does NOT apply simply because the overall role is AI-related. When coding on the basis of contextual specificity, confidence must be set to Medium or Low.

If a passage has NO AI specificity, do not extract it. The absence of AI content is itself a finding — capture it in the Rationale of the NONE entry if the entire ad has no AI content.

CRITICAL RULE: RESPONSIBILITIES VS. REQUIREMENTS

- GT codes (Layer 1) are assigned EXCLUSIVELY to passages from responsibilities, tasks, mandates, or accountabilities sections.
- SK codes (Layer 2) are assigned EXCLUSIVELY to passages from requirements, qualifications, or experience sections.
- When a passage is ambiguous, assign GT codes only if the language clearly implies organizational accountability rather than personal background.

CRITICAL RULE: DUAL CODING GT + SK

A single passage may receive both GT codes and SK codes simultaneously if it describes a governance responsibility (GT layer) AND implies a required competence (SK layer). Assign both when both dimensions are clearly present in the same passage.

CRITICAL RULE: PRIMARY CODE PRIORITIZATION

When a passage addresses multiple governance tensions, assign all codes that are clearly and substantively present. If one tension is clearly dominant and others are only marginally implied, assign only the dominant code and note the secondary tension in the Rationale. The threshold: would a trained human coder independently identify this code from the passage alone? If yes, assign it. If no, note it in the Rationale only.

CODEBOOK

LAYER 1 — GOVERNANCE TENSION CODES

Apply to: responsibilities / tasks / mandates sections only.

GT1 — Centralization of AI decision rights vs. decentralized experimentation

GT1.1 | Enterprise AI ownership & standards

Definition: Role given org-wide authority to define AI priorities, standards, policies, or decision rights.

Code when: Ad assigns enterprise-level ownership, standard-setting, or governance authority. NOT simple collaboration without authority.

Indicators: "own enterprise AI strategy", "define AI standards", "set AI policies", "enterprise AI operating model"

GT1.2 | Federated coordination across BUs

Definition: Role coordinates distributed AI across functions/BUs while preserving local autonomy.

Code when: Role integrates, aligns, or harmonizes AI work across units. Code CoE/hub-and-spoke. NOT if confined to one team.

Indicators: "coordinate AI across BUs", "center of excellence", "hub and spoke", "align central and local AI", "cross-functional team"

GT1.3 | Local experimentation enablement

Definition: Role enables BU-level experimentation, pilots, sandboxes, or decentralized AI use-case development.

Code when: Role explicitly supports local experimentation or bottom-up AI. NOT generic "innovation" alone.

Indicators: "enable experimentation", "support AI pilots", "incubate AI use cases", "sandbox", "empower product teams"

GT2 — Innovation and generativity vs. control, risk, and compliance

GT2.1 | AI opportunity creation & experimentation

Definition: Role identifies, initiates, or scales AI opportunities for innovation and value creation.

Code when: Ad emphasizes discovery, experimentation, use-case ID, or growth through AI. Exclude pure compliance tasks.

Indicators: "identify AI opportunities", "drive AI innovation", "build AI roadmap", "pilot and scale", "unlock business value", "AI portfolio"

GT2.2 | Responsible AI, risk & compliance

Definition: Role addresses ethical, legal, regulatory, privacy, or risk issues related to AI.

Code when: Ad mentions responsible AI, ethics, regulation, legal compliance, or risk mitigation related to AI.

Indicators: "responsible AI", "AI ethics", "regulatory compliance", "AI risk management", "privacy and security", "ethical AI framework"

GT2.3 | Governance frameworks & controls

Definition: Role designs or maintains formal governance mechanisms for AI oversight and control.

Code when: Role builds frameworks, committees, controls, review processes, or policies. NOT vague "best practices".

Indicators: "establish AI governance framework", "create review board", "define controls", "AI policy framework", "guidelines", "guardrails"

GT2.4 | AI enablement & change management

Definition: Role drives organizational AI adoption by managing the human and cultural side — enablement programs, internal champions, stakeholder engagement, friction reduction.

Code when: Ad assigns explicit responsibility for change management, adoption acceleration, workforce enablement, or internal advocacy related to AI. NOT generic "collaboration" without adoption mandate.

Indicators: "enablement", "AI literacy", "capacity building", "training", "change management", "champions", "ambassadors", "AI culture", "reduce friction"

GT3 — Technical and model-specific expertise vs. enterprise strategic alignment

GT3.1 | Deep AI/ML/GenAI expertise

Definition: Role requires substantial technical knowledge of AI methods, models, tools, or platforms as a governance dimension.

Code when: Ad asks for ML, GenAI, LLMs, data science, AI architecture, or tooling depth. NOT basic digital literacy.

Indicators: "machine learning", "LLMs", "deep learning", "AI architecture", "MLOps", "data science leadership"

GT3.2 | AI strategy & business value alignment

Definition: Role aligns AI with business strategy, competitive priorities, or value creation.

Code when: Ad links AI to enterprise strategy, transformation, or ROI. NOT generic leadership language without strategic alignment.

Indicators: "define AI strategy", "align AI with business goals", "value realization", "AI transformation", "strategic roadmap", "measurable value"

GT3.3 | Business-technical translation

Definition: Role translates between business needs and AI capabilities, or between executives and technical teams.

Code when: Ad explicitly frames role as interpreter, connector, or translator across business and technical domains.

Indicators: "translate business problems into AI", "bridge technical and nontechnical stakeholders", "communicate AI to executives"

GT4 — Data governance vs. model governance & lifecycle accountability

GT4.1 | Data governance responsibility

Definition: Role responsible for data quality, access, lineage, stewardship, privacy, or governance of inputs to AI.

Code when: Ad refers to governance of data assets feeding AI. NOT generic "data-driven decision making".

Indicators: "data governance", "data quality", "data stewardship", "data access controls", "data lineage", "trusted data"

GT4.2 | Model lifecycle governance

Definition: Role governs lifecycle of AI models from design and deployment to retraining, retirement, or updating.

Code when: Ad addresses lifecycle ownership of models.

Indicators: "model lifecycle management", "model deployment", "model retraining", "retire or update models"

GT4.3 | Model risk, validation & monitoring

Definition: Role accountable for validating, monitoring, testing, or assuring models in operation.

Code when: Ad mentions validation, monitoring, drift, robustness, model risk, or production assurance.

Indicators: "model validation", "performance monitoring", "model drift", "AI assurance", "model risk management", "risk assessment", "evaluation metrics"

GT5 — Algorithmic autonomy & opacity vs. human accountability

GT5.1 | Human oversight & final accountability

Definition: Role ensures humans remain answerable for AI-supported or AI-automated decisions.

Code when: Ad refers to oversight, escalation, approval rights, human-in-the-loop, or decision accountability for AI outcomes.

Indicators: "human oversight", "final decision authority", "escalation procedures", "human-in-the-loop"

GT5.2 | Explainability, transparency & documentation

Definition: Role makes AI systems understandable, traceable, or documentable.

Code when: Ad refers to explainability, transparency, interpretability, documentation, or audit trails of AI systems.

Indicators: "explainable AI", "transparency", "document model decisions", "traceability", "audit trail"

GT5.3 | Fairness, bias & safety protection

Definition: Role addresses harmful, unfair, unsafe, or discriminatory outcomes from AI use.

Code when: Ad mentions bias mitigation, fairness, safety, or harm prevention in relation to AI.

Indicators: "bias detection", "fair AI", "safe AI deployment", "trustworthy AI", "harm mitigation"

GT5.4 | Trustbuilding with stakeholders

Definition: Role builds interpersonal or organizational trust between stakeholders involved in AI use, inside and outside the organization.

Code when: Ad mentions trustbuilding, establishing trust, fostering trust, cultivating trust, building confidence, or building rapport in an AI-related context. NOT generic "trusted advisor" without AI context.

Indicators: "stakeholder trust building", "trusted advisor" (AI context only), "establish credibility", "build confidence in AI"

LAYER 2 — SKILL/COMPETENCE CODES (Todd et al., 1995)

Apply to: requirements / qualifications / experience sections only.

SK1 | Technical IT/AI knowledge

Definition: Required technical competence in AI, ML, data science, cloud, engineering, tooling, or infrastructure.

Code when: Concrete technical knowledge or tool requirements related to AI.

Indicators: "Python", "cloud AI platforms", "data engineering", "MLOps stack", "AI architecture", "LLMs", "machine learning"

SK2 | Business/domain knowledge

Definition: Required industry, functional, or business-process knowledge to identify and shape AI use cases.

Code when: Ad asks for business acumen, domain expertise, process knowledge, or commercial understanding.

Indicators: "strong business acumen", "industry expertise", "understanding of operations", "commercial orientation", "domain knowledge"

SK3 | Systems/integration knowledge

Definition: Skills for connecting AI into enterprise systems, processes, platforms, workflows, and architectures.

Code when: Ad stresses integration of AI into existing IS landscape.

Indicators: "enterprise architecture", "systems integration", "platform integration",
"connect AI into core systems"

OUTPUT FORMAT

Return a JSON array. Each element represents one coded passage. Use this structure:

```
{  
  "Seg_ID": "<Ad_ID>_S<sequential number starting at 01>",  
  "Ad_ID": "<provided Ad_ID>",  
  "Role_Family": "<provided Role_Family>",  
  "Text_Segment": "<exact extracted text from the job ad>",  
  "GT_Code": "<semicolon-separated GT codes, or NONE>",  
  "SK_Code": "<semicolon-separated SK codes, or NONE>",  
  "Inductive_Code": "<short free-text label if new pattern, else null>",  
  "Confidence": "<HIGH / MEDIUM / LOW>",  
  "Rationale": "<2-4 sentences: which words triggered which codes, why GT or SK  
applies, and if contextual specificity was used>"  
}
```

Important output rules:

- Return ONLY a valid JSON array. No preamble, no explanation outside the JSON.
- Generate Seg_IDs sequentially per ad: {ad_id}_S01, {ad_id}_S02 etc.
- If the entire ad has no AI-relevant content, return a single element with GT_Code: NONE, SK_Code: NONE, Text_Segment: null, and a Rationale explaining the absence.
- Always write the Rationale in English, even for German-language ads.
- Extract Text_Segment verbatim from the ad — do not paraphrase or summarize.

- Do not extract boilerplate: salary, benefits, equal opportunity statements, application instructions.

EXAMPLES

Example A — Single responsibility passage:

Input contains: "Establish procedures and frameworks for responsible AI monitoring and testing."

Output element:

```
{{"Seg_ID": "JOB_0004_S01", "Ad_ID": "JOB_0004", "Role_Family": "AI Manager",  
  "Text_Segment": "Establish procedures and frameworks for responsible AI monitoring  
and testing.", "GT_Code": "GT2.3", "SK_Code": "NONE", "Inductive_Code": null,  
  "Confidence": "HIGH", "Rationale": "The passage assigns responsibility for building  
formal governance mechanisms — 'procedures and frameworks' — for AI oversight,  
mapping to GT2.3. 'Responsible AI monitoring and testing' provides lexical AI specificity.  
Responsibilities section, so no SK code applies."}}
```

Example B — Dual GT + SK passage:

Input contains: "Design and deliver AI solutions aligned to business priorities, enterprise architecture, and trusted data."

Output element:

```
{{"Seg_ID": "JOB_0637_S03", "Ad_ID": "JOB_0637", "Role_Family": "AI Lead",  
  "Text_Segment": "Design and deliver AI solutions aligned to business priorities,  
enterprise architecture, and trusted data.", "GT_Code": "GT3.2; GT4.1", "SK_Code": "SK3",  
  "Inductive_Code": null, "Confidence": "HIGH", "Rationale": "GT3.2: 'aligned to business  
priorities' links AI delivery to enterprise strategy. GT4.1: 'trusted data' signals data  
governance as input to AI. SK3: 'enterprise architecture' implies systems/integration  
knowledge. The passage describes a responsibility that simultaneously implies a  
required competence, justifying dual coding."}}
```

Example C — Zero codes:

Output: [{"Seg_ID": "JOB_0005_S01", "Ad_ID": "JOB_0005", "Role_Family": "CTO", "Text_Segment": null, "GT_Code": "NONE", "SK_Code": "NONE", "Inductive_Code": null, "Confidence": "HIGH", "Rationale": "No AI-specific content found in the entire job ad. The posting describes general software engineering leadership without any reference to AI, ML, GenAI, or direct equivalents. Zero codes assigned."}]

INPUT

Ad_ID: {ad_id}

Role_Family: {role_family}

Language: {language}

Full job advertisement text:

{full_text}"""

=====

HELPER FUNCTIONS

=====

def load_dataset(filepath):

"""Load all job ads from the Excel file."""

wb = openpyxl.load_workbook(filepath, data_only=True)

all_ads = []

for sheet_name in SHEETS_TO_PROCESS:

if sheet_name not in wb.sheetnames:

print(f" WARNING: Sheet '{sheet_name}' not found, skipping.")

```
continue
```

```
ws = wb[sheet_name]
```

```
headers = [cell.value for cell in ws[1]]
```

```
# Find column indices
```

```
col_map = {h: i for i, h in enumerate(headers) if h}
```

```
required = ['id', 'job_title_category', 'language', 'full_text']
```

```
for req in required:
```

```
    if req not in col_map:
```

```
        print(f" WARNING: Column '{req}' not found in sheet '{sheet_name}'")
```

```
        continue
```

```
rows_loaded = 0
```

```
for row in ws.iter_rows(min_row=2, values_only=True):
```

```
    if not any(row):
```

```
        continue
```

```
ad = {
```

```
    'sheet': sheet_name,
```

```
    'id': row[col_map.get('id', 0)],
```

```
    'role_family': row[col_map.get('job_title_category', 4)],
```

```
    'language': row[col_map.get('language', 12)],
```

```
    'full_text': row[col_map.get('full_text', 13)],
```

```
    'company': row[col_map.get('company_name', 5)] if 'company_name' in col_map  
else None,
```

```
    'industry': row[col_map.get('industry', 7)] if 'industry' in col_map else None,
```

```
        'country': row[col_map.get('location_country', 9)] if 'location_country' in col_map
    else None,
}
```

```
if SKIP_EMPTY and (not ad['full_text'] or len(str(ad['full_text'])) < 100):
    continue
```

```
all_ads.append(ad)
rows_loaded += 1
```

```
print(f" Loaded {rows_loaded} ads from sheet '{sheet_name}'")
```

```
return all_ads
```

```
def call_api(client, ad):
```

```
    """Send a single ad to the API and return parsed coding results."""
```

```
    prompt = USER_PROMPT_TEMPLATE.format(
        ad_id=ad['id'],
        role_family=ad['role_family'],
        language=ad['language'] or 'unknown',
        full_text=str(ad['full_text'])
    )
```

```
    response = client.messages.create(
        model=MODEL,
        max_tokens=MAX_TOKENS,
        system=SYSTEM_PROMPT,
        messages=[{"role": "user", "content": prompt}]
```


)

```
raw_text = response.content[0].text.strip()
```

```
# Clean up common JSON formatting issues
```

```
if raw_text.startswith("` ` ` json"):
```

```
    raw_text = raw_text[7:]
```

```
if raw_text.startswith("` ` `"):
```

```
    raw_text = raw_text[3:]
```

```
if raw_text.endswith("` ` `"):
```

```
    raw_text = raw_text[:-3]
```

```
raw_text = raw_text.strip()
```

```
# Parse JSON
```

```
results = json.loads(raw_text)
```

```
# Ensure it's a list
```

```
if isinstance(results, dict):
```

```
    results = [results]
```

```
return results, response.usage.input_tokens, response.usage.output_tokens
```

```
def setup_output_workbook():
```

```
    """Create and style the output Excel workbook."""
```

```
    wb = openpyxl.Workbook()
```

```
    ws = wb.active
```

```
    ws.title = "Coding Results"
```

Styles

NAVY = PatternFill('solid', start_color='1F3864')

LIGHT_BLUE = PatternFill('solid', start_color='D6E4F0')

LIGHT_GREEN = PatternFill('solid', start_color='E2EFDA')

LIGHT_YELLOW = PatternFill('solid', start_color='FFF2CC')

WHITE = PatternFill('solid', start_color='FFFFFF')

thin = Side(style='thin', color='BFBFBF')

BORDER = Border(left=thin, right=thin, top=thin, bottom=thin)

WRAP = Alignment(wrap_text=True, vertical='top')

CENTER = Alignment(horizontal='center', vertical='center', wrap_text=True)

Title row

ws.merge_cells('A1:N1')

ws['A1'] = 'LLM CODING OUTPUT — AI Leadership Role Content Analysis'

ws['A1'].font = Font(name='Arial', bold=True, size=13, color='FFFFFF')

ws['A1'].fill = NAVY

ws['A1'].alignment = Alignment(horizontal='center', vertical='center')

ws.row_dimensions[1].height = 26

Column headers

headers = [

 ('Seg_ID', 14),

 ('Ad_ID', 14),

 ('Role_Family', 18),

 ('Sheet', 16),

 ('Company', 22),

 ('Industry', 24),

 ('Country', 10),

```

('Language', 10),
('Text_Segment', 60),
('GT_Code', 18),
('SK_Code', 12),
('Inductive_Code', 20),
('Confidence', 12),
('Rationale', 50),
]

```

```

for j, (label, width) in enumerate(headers, 1):
    c = ws.cell(row=2, column=j, value=label)
    c.font = Font(name='Arial', bold=True, size=9, color='FFFFFF')
    c.fill = PatternFill('solid', start_color='2E75B6')
    c.alignment = CENTER
    c.border = BORDER
    ws.column_dimensions[get_column_letter(j)].width = width

ws.row_dimensions[2].height = 24
ws.freeze_panes = 'A3'

return wb, ws

```

```

def write_results(ws, results, ad, row_num):
    """Write coding results for one ad to the worksheet."""
    thin = Side(style='thin', color='BFBFBF')
    BORDER = Border(left=thin, right=thin, top=thin, bottom=thin)
    WRAP = Alignment(wrap_text=True, vertical='top')
    WHITE = PatternFill('solid', start_color='FFFFFF')

```

```
LIGHT_GREEN = PatternFill('solid', start_color='E2EFDA')
LIGHT_YELLOW = PatternFill('solid', start_color='FFF2CC')
LIGHT_BLUE = PatternFill('solid', start_color='D6E4F0')
```

```
# Confidence color coding
```

```
CONF_COLORS = {
    'HIGH': PatternFill('solid', start_color='E2EFDA'),
    'MEDIUM': PatternFill('solid', start_color='FFF2CC'),
    'LOW': PatternFill('solid', start_color='FCE4D6'),
}
```

```
for seg in results:
```

```
    confidence = str(seg.get('Confidence', '')).upper()
    conf_fill = CONF_COLORS.get(confidence, WHITE)
```

```
    row_data = [
        seg.get('Seg_ID', ''),
        seg.get('Ad_ID', ad['id']),
        seg.get('Role_Family', ad['role_family']),
        ad.get('sheet', ''),
        ad.get('company', ''),
        ad.get('industry', ''),
        ad.get('country', ''),
        ad.get('language', ''),
        seg.get('Text_Segment', ''),
        seg.get('GT_Code', ''),
        seg.get('SK_Code', ''),
        seg.get('Inductive_Code', ''),
        seg.get('Confidence', ''),
```

```
seg.get('Rationale', ''),  
]
```

```
for j, val in enumerate(row_data, 1):  
    c = ws.cell(row=row_num, column=j, value=val if val is not None else '')  
    c.border = BORDER  
    c.alignment = WRAP  
    c.font = Font(name='Arial', size=9)
```

```
# Color coding by column type
```

```
if j in [10, 11]: # GT_Code, SK_Code
```

```
    c.fill = LIGHT_GREEN
```

```
elif j == 13: # Confidence
```

```
    c.fill = conf_fill
```

```
elif j == 14: # Rationale
```

```
    c.fill = LIGHT_YELLOW
```

```
elif j == 12: # Inductive_Code
```

```
    c.fill = LIGHT_BLUE if val else WHITE
```

```
else:
```

```
    c.fill = WHITE
```

```
ws.row_dimensions[row_num].height = 60
```

```
row_num += 1
```

```
return row_num
```

```
def add_log_sheet(wb, log_entries):
```

```
    """Add a processing log sheet to the workbook."""
```

```
ws = wb.create_sheet("Processing Log")
ws.append(['Ad_ID', 'Role_Family', 'Status', 'Segments_Found',
          'Input_Tokens', 'Output_Tokens', 'Error'])
```

```
for entry in log_entries:
```

```
    ws.append([
        entry.get('ad_id', ''),
        entry.get('role_family', ''),
        entry.get('status', ''),
        entry.get('segments', 0),
        entry.get('input_tokens', 0),
        entry.get('output_tokens', 0),
        entry.get('error', ''),
    ])
```

```
ws.column_dimensions['A'].width = 14
ws.column_dimensions['B'].width = 18
ws.column_dimensions['C'].width = 10
ws.column_dimensions['D'].width = 16
ws.column_dimensions['E'].width = 14
ws.column_dimensions['F'].width = 14
ws.column_dimensions['G'].width = 50
```

```
# =====
```

```
# MAIN CODING LOOP
```

```
# =====
```

```
def main():
```

```
print("=" * 60)

print("AI Leadership Role Content Analysis — LLM Coding Run")

print("=" * 60)

print(f"Input file: {INPUT_FILE}")

print(f"Output file: {OUTPUT_FILE}")

print(f"Model: {MODEL}")

print()


# Load data

print("Loading dataset...")

ads = load_dataset(INPUT_FILE)

total = len(ads)

print(f"Total ads to process: {total}")

print()


if total == 0:

    print("ERROR: No ads loaded. Check INPUT_FILE path and SHEETS_TO_PROCESS.")

    return


# Initialize API client

client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)


# Setup output file

wb, ws = setup_output_workbook()

current_row = 3


# Tracking

log_entries = []

total_input_tokens = 0
```

```

total_output_tokens = 0

errors = []


# Resume support
start_index = RESUME_FROM_INDEX

if start_index > 0:

    print(f"Resuming from index {start_index}...")


# Main loop
for i, ad in enumerate(ads[start_index:], start=start_index):

    ad_id = ad['id'] or f"UNKNOWN_{i}"

    role_family = ad['role_family'] or 'Unknown'


    print(f"[{i+1:4d}/{total}] {ad_id} | {role_family} | "
          f"{str(ad['full_text'])[:50].replace(chr(10),' ')}...")


    try:

        results, in_tok, out_tok = call_api(client, ad)


        total_input_tokens += in_tok

        total_output_tokens += out_tok


        current_row = write_results(ws, results, ad, current_row)


    log_entries.append({

        'ad_id': ad_id,

        'role_family': role_family,

        'status': 'OK',

        'segments': len(results),

```



```

        'input_tokens': in_tok,
        'output_tokens': out_tok,
        'error': '',
    })

```

```

print(f"    → {len(results)} segments coded | "
      f"{in_tok} in / {out_tok} out tokens")

```

```

except json.JSONDecodeError as e:

```

```

    msg = f"JSON parse error: {e}"
    print(f"    ERROR: {msg}")
    errors.append((ad_id, msg))
    log_entries.append({
        'ad_id': ad_id, 'role_family': role_family,
        'status': 'ERROR', 'segments': 0,
        'input_tokens': 0, 'output_tokens': 0, 'error': msg
    })

```

```

except anthropic.RateLimitError:

```

```

    print("    Rate limit hit — waiting 60 seconds...")
    time.sleep(60)
    # Retry once
    try:
        results, in_tok, out_tok = call_api(client, ad)
        total_input_tokens += in_tok
        total_output_tokens += out_tok
        current_row = write_results(ws, results, ad, current_row)
        log_entries.append({
            'ad_id': ad_id, 'role_family': role_family,

```

```

        'status': 'OK (retry)', 'segments': len(results),
        'input_tokens': in_tok, 'output_tokens': out_tok, 'error': ''
    })
except Exception as e2:
    msg = f"Retry failed: {e2}"
    print(f"    ERROR on retry: {msg}")
    errors.append((ad_id, msg))
    log_entries.append({
        'ad_id': ad_id, 'role_family': role_family,
        'status': 'ERROR', 'segments': 0,
        'input_tokens': 0, 'output_tokens': 0, 'error': msg
    })

except Exception as e:
    msg = f"{type(e).__name__}: {e}"
    print(f"    ERROR: {msg}")
    errors.append((ad_id, msg))
    log_entries.append({
        'ad_id': ad_id, 'role_family': role_family,
        'status': 'ERROR', 'segments': 0,
        'input_tokens': 0, 'output_tokens': 0, 'error': msg
    })

# Save periodically every 25 ads
if (i + 1) % 25 == 0:
    add_log_sheet(wb, log_entries)
    wb.save(OUTPUT_FILE)
    print(f" [Checkpoint saved at ad {i+1}]")
    if 'Processing Log' in wb.sheetnames:

```

```

del wb['Processing Log']

# Delay to avoid rate limiting
time.sleep(API_DELAY)

# Final save
add_log_sheet(wb, log_entries)
wb.save(OUTPUT_FILE)

# Summary
print()
print("=" * 60)
print("RUN COMPLETE")
print("=" * 60)
print(f"Ads processed:   {total - start_index}")
print(f"Errors:           {len(errors)}")
print(f"Total input tokens: {total_input_tokens:,}")
print(f"Total output tokens:{total_output_tokens:,}")
est_cost = (total_input_tokens / 1_000_000 * 3.0) + \
            (total_output_tokens / 1_000_000 * 15.0)
print(f"Estimated cost:   ~${est_cost:.2f} USD")
print(f"Output saved to:   {OUTPUT_FILE}")

if errors:
    print(f"\nFailed ads ({len(errors)}):")
    for ad_id, msg in errors:
        print(f" {ad_id}: {msg}")

```

```
if __name__ == "__main__":  
    main()
```